■ Python Concurrency & Parallelism

Complete Interview Preparation Notes

Threads • Processes • GIL • asyncio • Lock • Queue

■ Topic	■ Description
Lecture 1	Concurrency vs Parallelism — Concept + Basic Code
Lecture 2	GIL (Global Interpreter Lock) — In Action + Bypass
Lecture 3	Threading Deep Dive — Lock, I/O ops, Args
Lecture 4	Multiprocessing — Queue, Value, Real World Use
Interview Q&A	Top Interview Questions with Perfect Answers
Quick Revision	One-liner cheatsheet for last minute prep

■ LECTURE 1 — Concurrency vs Parallelism

■ Concurrency kya hai?

Definition: Concurrency means handling multiple tasks at once by switching between them so rapidly that it appears simultaneous — but only ONE task is actually running at any given moment.

Real Life Example: Ek waiter 3 tables handle karta hai — Table 1 ka order leta hai, phir Table 2, phir Table 3. Ek waqt mein sirf ek table pe hai, lekin switching itni fast hai ki sab ko lagta hai served ho rahe hain.

Python Module: `threading` — CPU switching rapidly between threads on ONE core.

■ Parallelism kya hai?

Definition: Parallelism means truly executing multiple tasks at the EXACT same time — each task runs on its own dedicated CPU core simultaneously.

Real Life Example: Teen alag waiters, teen alag tables — teeno ek waqt mein kaam kar rahe hain. Koi switching nahi, sach mein ek saath!

Python Module: `multiprocessing` — Multiple CPU cores, truly parallel execution.

■ Concurrency vs Parallelism — Diagram:

■■ Parallelism ka Lazy Worker Problem:

```
Core 1 : [==========] ■ Done in 3 sec
Core 2 : [==========] ■ Done in 3 sec
Core 3 : [=====          ] ■ Still running...

→ Jab tak Core 3 khatam nahi → Final result BLOCKED!
→ OS ne Core 3 ko aur kaam de diya hoga
→ Result combine karna bhi extra overhead!
```

■ Threading Code (Concurrency):

```
import threading, time
```

```
def take_orders():
    for i in range(1, 4):
        print(f'Taking order {i}')
        time.sleep(2)   # DB call / file read simulate

def brew_chai():
    for i in range(1, 4):
        print(f'Brewing chai {i}')
        time.sleep(3)

order_thread = threading.Thread(target=take_orders)  # Create
brew_thread  = threading.Thread(target=brew_chai)    # Create

order_thread.start()   # Start karo
brew_thread.start()    # Yeh bhi start

order_thread.join()    # Wait karo khatam hone tak
brew_thread.join()     # Yeh bhi wait

print('All done!')
```

■ Multiprocessing Code (Parallelism):

```
from multiprocessing import Process
import time

def brew_chai(name):
    print(f'Start: {name}')
    time.sleep(3)   # Heavy CPU work
    print(f'Done: {name}')

if __name__ == '__main__':   # MANDATORY for multiprocessing!
    chai_makers = [
        Process(target=brew_chai, args=(f'Chai {i+1}',))
        for i in range(3)    # 3 separate processes
    ]
    for p in chai_makers: p.start()  # All start together
    for p in chai_makers: p.join()   # Wait for all
    print('All served!')
```

■ Comparison Table:

Feature	Threading (Concurrency)	Multiprocessing (Parallelism)
CPU Cores	1 core	Multiple cores
Execution	Interleaved (switching)	Truly simultaneous
Memory	Shared memory	Separate memory
Best For	I/O bound tasks	CPU bound tasks
Module	threading	multiprocessing

Main Guard	Not required	if __name__=='__main__' REQUIRED
Speed (I/O)	■■■■■ Excellent	■■ Overkill
Speed (CPU)	■■ Blocked by GIL	■■■■■ Excellent

■ LECTURE 2 — GIL (Global Interpreter Lock)

■ GIL kya hai?

Full Form: Global Interpreter Lock

Definition: GIL ek mutex (Mutually Exclusive Lock) hai jo Python mein sirf EK thread ko ek waqt mein Python bytecode execute karne deta hai — chahe multiple threads hon.

Kyun zaroori hai? CPython ka memory management thread-safe nahi hai. Agar do threads ek saath same memory location change karein → Race Condition hoga.

Real Life Example: Chai counter — chahe kitne bhi baristas hon, ek waqt mein sirf ek order process hoga!

■ Race Condition kya hota hai?

```
Memory mein value = 4

Thread 1: 'Main 4 ko 5 karna chahta hoon'
Thread 2: 'Main 4 ko 3 karna chahta hoon'

Dono ek saath try karein toh?
→ Final value kya hogi? 5? 3? Corrupt?
→ Yahi hai RACE CONDITION!

GIL ka kaam:
→ Thread 1 ko MUTEX LOCK milta hai
→ Thread 2 WAIT karta hai
→ Thread 1 kaam karta hai → Lock release karta hai
→ Thread 2 ko Lock milta hai → Ab kaam karta hai
```

■ GIL — Threading Slow kyon?

```
import threading, time

def brew_chai():
    print(f'{threading.current_thread().name} started...')
    count = 0
    for _ in range(10_000_000):  # CPU heavy task
        count += 1
    print(f'{threading.current_thread().name} finished!')

t1 = threading.Thread(target=brew_chai, name='Barista 1')
t2 = threading.Thread(target=brew_chai, name='Barista 2')

start = time.time()
t1.start(); t2.start()
t1.join(); t2.join()
print(f'Time: {time.time()-start:.2f}s')

# Result: ~5 seconds (SLOW!)
```

```
# Kyun? GIL dono threads ko simultaneously nahi chalata
```

■ GIL Bypass — Multiprocessing se!

```
from multiprocessing import Process
import time

def crunch_number():
    count = 0
    for _ in range(10_000_000):
        count += 1

if __name__ == '__main__':
    start = time.time()
    p1 = Process(target=crunch_number)
    p2 = Process(target=crunch_number)
    p1.start(); p2.start()
    p1.join(); p2.join()
    print(f'Time: {time.time()-start:.2f}s')

# Result: ~2.8 seconds (FAST!)
# Kyun? Har process ka APNA GIL → True Parallelism!
```

■ GIL Summary:

- **Threading + CPU task** = SLOW (~5s) → GIL blocks both threads
- **Multiprocessing + CPU task** = FAST (~2.8s) → Each process has own GIL
- **Threading + I/O task** = FAST → GIL released during I/O wait!

Interview Gold: Python mein threading se true parallelism nahi milti CPU tasks ke liye. Isliye CPU-bound tasks ke liye hamesha multiprocessing use karo!

■ LECTURE 3 — Threading Deep Dive

■ Thread ke Key Methods:

Method / Concept	Kya karta hai	Example
threading.Thread()	Thread object banao	t = threading.Thread(target=func)
.start()	Thread ka execution shuru karo	t.start()
.join()	Wait karo jab tak thread khatam na ho	t.join()
target=	Kaunsa function run karna hai	target=my_function
args=	Function ke arguments (TUPLE mein!)	args=('masala', 2)
name=	Thread ka naam dena	name='Barista 1'
threading.Lock()	Thread-safe lock banana	lock = threading.Lock()
with lock:	Lock leke safely kaam karo	with lock: counter += 1

■ Args Pass Karna — Tuple!

```
import threading, time

def prepare_chai(type_, wait_time):
    print(f'{type_} chai brewing...')
    time.sleep(wait_time)
    print(f'{type_} chai ready!')

# Args TUPLE mein pass hote hain (comma ZAROOR lagao!)
t1 = threading.Thread(target=prepare_chai, args=('Masala', 2))
t2 = threading.Thread(target=prepare_chai, args=('Ginger', 3))

t1.start()
t2.start()
t1.join()
t2.join()
# Output: Masala aur Ginger dono brewing ek saath shuru!
```

■ Threading kahan SHINE karta hai — I/O Operations

Threading best hai jab task WAITING karta hai:
■ Database queries ■ API/Web requests ■ File read/write ■ Network operations

Kyun? Jab thread I/O operation pe wait karta hai → GIL release hota hai → Doosra thread kaam karta hai! No memory conflict = Fast execution!

```
import threading, requests, time

def download(url):
    print(f'Downloading: {url}')
```

```
    response = requests.get(url)   # I/O wait hoga
    print(f'Done! {len(response.content)} bytes')

urls = [
    'https://httpbin.org/image/jpeg',
    'https://httpbin.org/image/png',
    'https://httpbin.org/image/svg',
]

start = time.time()
threads = []
for url in urls:
    t = threading.Thread(target=download, args=(url,))
    t.start()
    threads.append(t)

for t in threads:
    t.join()
print(f'All done in {time.time()-start:.2f}s')  # Fast!
```

■ Thread Lock — Race Condition se Bachao

```
import threading

counter = 0
lock = threading.Lock()  # Mutex lock banao

def increment():
    global counter
    for _ in range(100_000):
        with lock:           # Lock leke safely modify karo
            counter += 1     # Thread-safe operation

# 10 threads create karo
threads = [threading.Thread(target=increment) for _ in range(10)]
for t in threads: t.start()
for t in threads: t.join()

print(f'Final counter: {counter}')  # 1,000,000 ■

# BINA LOCK ke: counter kabhi kabhi galat value deta hai!
# Race Condition: Thread 1 aur Thread 2 dono counter=5 padhte hain
#                 dono +1 karte hain → dono 6 likhte hain
#                 Expected 7 tha, mila 6! ■
```

■ LECTURE 4 — Multiprocessing Deep Dive

■■ Process ke Key Methods:

Method / Concept	Kya karta hai	Example
Process()	Alag process banao	p = Process(target=func)
.start()	Process shuru karo (apne core pe!)	p.start()
.join()	Wait karo process khatam hone tak	p.join()
target=	Kaunsa function run karna hai	target=cpu_task
args=	Function args (TUPLE)	args=('value',)
if __name__=='__main__'	ZAROORI — entry point batata hai	Always use this!
Queue()	Processes ke beech data share karo	p = Queue()
q.put()	Queue mein data daalo	q.put('result')
q.get()	Queue se data nikalo	data = q.get()
Value('i', 0)	Shared integer across processes	counter = Value('i', 0)
counter.get_lock()	Value ka built-in lock	with counter.get_lock():

■■ Thread vs Process — CPU Speed Comparison

```
# ■■ THREADING (CPU task) — SLOW ■■■■■■■■■■■■■■■■■■■■■■■■
import threading, time
def cpu_heavy():
    total = 0
    for i in range(10**7): total += i

start = time.time()
threads = [threading.Thread(target=cpu_heavy) for _ in range(2)]
for t in threads: t.start()
for t in threads: t.join()
print(f'Threading: {time.time()-start:.2f}s')  # ~1.0s

# ■■ MULTIPROCESSING (CPU task) — FAST ■■■■■■■■■■■■■■■■■■■■
from multiprocessing import Process
start = time.time()
processes = [Process(target=cpu_heavy) for _ in range(2)]
for p in processes: p.start()
for p in processes: p.join()
print(f'Multiprocessing: {time.time()-start:.2f}s')  # ~0.35s

# Result: Multiprocessing ~3x FASTER for CPU tasks!
```

■ Queue — Process ke beech Communication

Problem: Processes ka memory alag hota hai — direct data share nahi ho sakta!
Solution: Queue = Common post box jo sab processes use kar sakte hain

Process 1 → q.put('data') → Queue → Process 2 → q.get()

```
from multiprocessing import Process, Queue

def prepare_chai(q):
    # Kaam karo aur result queue mein daalo
    q.put('Masala chai is ready!')

if __name__ == '__main__':
    q = Queue()   # Shared queue banao
    p = Process(target=prepare_chai, args=(q,))
    p.start()
    p.join()
    result = q.get()          # Queue se result nikalo
    print(result)             # 'Masala chai is ready!'
```

■ Value — Shared Counter across Processes

```
from multiprocessing import Process, Value

def increment(counter):
    for _ in range(100_000):
        with counter.get_lock():  # Built-in lock auto milta hai!
            counter.value += 1

if __name__ == '__main__':
    counter = Value('i', 0)   # 'i' = integer, 0 = initial value
    processes = [
        Process(target=increment, args=(counter,))
        for _ in range(4)     # 4 processes
    ]
    for p in processes: p.start()
    for p in processes: p.join()
    print(f'Final: {counter.value}')  # 400,000 ■
```

■ Real World Use Cases for Multiprocessing:

- **Batch Image Processing:** 100 images hain → 4 cores pe 25-25 assign karo → 4x faster!
- **ML Model Training:** PyTorch/TensorFlow multiprocessing use karta hai internally
- **Data Pipeline:** Large CSV files ko chunks mein process karo parallel mein
- **Video Rendering:** Alag alag frames alag cores pe process karo

■ TOP INTERVIEW QUESTIONS & ANSWERS

Q1. What is the difference between Concurrency and Parallelism?

Concurrency means dealing with multiple tasks at once — only ONE task runs at any moment, but CPU switches so rapidly it appears simultaneous. Parallelism means truly executing multiple tasks at the EXACT same time using separate CPU cores. In Python: threading = concurrency, multiprocessing = parallelism. One-liner: Concurrency = DEALING with many tasks. Parallelism = DOING many tasks.

Q2. What is GIL (Global Interpreter Lock)?

GIL is a mutex in CPython that allows only ONE thread to execute Python bytecode at a time. It exists because CPython's memory management is not thread-safe. Without GIL, two threads could modify the same memory simultaneously causing corruption (race condition). Impact: threading cannot achieve true parallelism for CPU-bound tasks in Python. Workaround: Use multiprocessing — each process has its own GIL.

Q3. When would you use Threading vs Multiprocessing?

Use THREADING for I/O-bound tasks: database calls, API requests, file read/write, network operations. During I/O wait, GIL is released so other threads run — effective concurrency! Use MULTIPROCESSING for CPU-bound tasks: image processing, ML training, large calculations. Each process gets its own GIL → true parallelism → significantly faster. Golden Rule: Task is WAITING → Threading. Task is COMPUTING → Multiprocessing.

Q4. What is a Race Condition? How do you prevent it?

Race condition occurs when two threads simultaneously read and modify the same memory location, leading to incorrect results. Example: Thread 1 and Thread 2 both read counter=5, both add 1, both write 6 — but expected result was 7! Prevention: Use threading.Lock() — only one thread can hold the lock at a time. with lock: ensures thread-safe access to shared data.

Q5. What is .join() and why is it important?

join() blocks the calling (main) thread until the thread/process it's called on terminates. Without join(), the main program might finish before threads complete, giving incomplete results. Example: t1.join() means 'wait here until t1 finishes, then proceed'. In multiprocessing: same concept — p.join() waits for the process to complete.

Q6. How do processes communicate with each other?

Processes have separate memory spaces — they cannot directly share variables. Solutions: Queue (multiprocessing.Queue) — works like a shared post box. Process puts data with q.put(), another process retrieves with q.get(). Value (multiprocessing.Value) — shared primitive type with built-in lock via counter.get_lock(). Pipes — for bidirectional communication between exactly two processes.

Q7. Why must we use if __name__ == '__main__' in multiprocessing?

Without this guard, when multiprocessing spawns a new process, the new process imports the script and re-executes all top-level code — potentially creating infinite process spawning! The if __name__ == '__main__' guard ensures process-creation code only runs in the main script, not in spawned child processes. NOT required for threading (threads don't re-import the script).

Q8. What is asyncio and how does it differ from threading?

asyncio is Python's built-in library for asynchronous programming using async/await syntax. It achieves concurrency in a single thread using an event loop — no thread switching overhead! Threading: OS decides when to switch threads (preemptive). asyncio: YOU decide when to yield control using await (cooperative). asyncio is ideal for I/O-heavy applications. FastAPI is built entirely on asyncio.

■ QUICK REVISION CHEATSHEET

■ One-Liner Definitions:

```
Concurrency    = One core, multiple tasks, fast switching
Parallelism    = Multiple cores, multiple tasks, truly simultaneous
GIL            = Python lock — only 1 thread runs at a time
Race Condition = 2 threads corrupt same memory simultaneously
Mutex / Lock   = Only 1 thread can access protected code at a time
Thread         = Lightweight unit inside a process, shares memory
Process        = Independent program with its own memory
join()         = Wait until thread/process finishes
start()        = Begin execution of thread/process
Queue          = Shared data structure between processes
asyncio        = Single-threaded concurrency using event loop
I/O Bound      = Task spends time WAITING (DB, API, file)
CPU Bound      = Task spends time COMPUTING (math, ML, video)
```

■ When to Use What:

Situation	Best Choice	Python Tool
Database calls / API requests	Concurrency	threading / asyncio
File read/write operations	Concurrency	threading / asyncio
Web scraping multiple URLs	Concurrency	threading
FastAPI web server	Concurrency	asyncio
Video / Image processing	Parallelism	multiprocessing
ML model training	Parallelism	multiprocessing
Large math calculations	Parallelism	multiprocessing
Multiple process coordination	Parallelism	multiprocessing + Queue
Shared counter (thread-safe)	Concurrency	threading + Lock
Shared counter (process-safe)	Parallelism	multiprocessing + Value

■ Interview Script — Bol Diya Toh Job Pakki!

"Concurrency and Parallelism are both techniques to handle multiple tasks, but they differ fundamentally.

Concurrency means a single CPU core handles multiple tasks by switching between them rapidly. Only one task runs at any instant. In Python, we use threading or asyncio for this.

Parallelism means multiple CPU cores genuinely execute multiple tasks at the exact same time. In Python, we use multiprocessing for this.

An important Python-specific point — Python has a Global Interpreter Lock (GIL) which prevents multiple threads from executing simultaneously. So for CPU-heavy tasks, threading does NOT give true parallelism — we must use multiprocessing.

For I/O-bound tasks like DB calls or API requests, threading is ideal because the GIL is released during I/O waits. For CPU-bound tasks like video processing or ML training, multiprocessing is the choice."

■■ Common Mistakes to Avoid:

```
■ WRONG: args=('value')      ← Not a tuple!
■ RIGHT: args=('value',)     ← Comma makes it tuple

■ WRONG: Missing if __name__=='__main__' in multiprocessing
■ RIGHT: Always wrap process-creation code in main guard

■ WRONG: Using threading for heavy CPU computation
■ RIGHT: Use multiprocessing for CPU-bound tasks

■ WRONG: Sharing variables directly between processes
■ RIGHT: Use Queue or Value for inter-process communication

■ WRONG: Forgetting .join() — main exits before threads finish
■ RIGHT: Always call .join() after .start()
```

■ Golden Rule to Remember:

"If your task is **WAITING** → Use **Concurrency** (threading/asyncio)"

"If your task is **COMPUTING** → Use **Parallelism** (multiprocessing)"